

# INF01151 – Sistemas Operacionais II

---

## Aula 22 - Exclusão mútua (distribuída)

Prof. Alberto Egon Schaeffer Filho

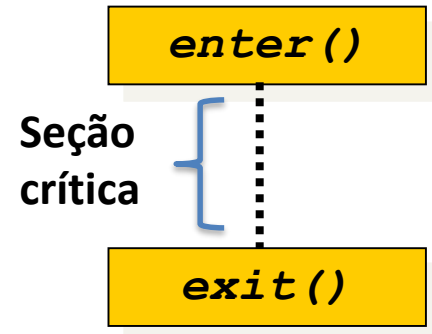
---

# Introdução

- Sincronização de acesso a dados e de controle são problemas clássicos em sistemas operacionais
  - Oriundo da necessidade de processos compartilharem recursos
  - Ocorre também em sistemas distribuídos... dessa forma são necessários mecanismos para:
    - Exclusão mútua em sistemas distribuídos
    - Evitar interferência e garantir consistência
- Em sistemas centralizados
  - Resolvido com emprego de mutexes, variáveis de condição e semáforos (existência de memória comum entre processos)
- Em sistemas distribuídos
  - Problema deve ser obrigatoriamente resolvido através de troca de mensagens (não há memória comum entre processos)
    - Caso trivial: recurso é gerenciado por um servidor que provê exclusão mútua
    - **Caso geral: necessita de um mecanismo separado para exclusão mútua**

# Algoritmos para exclusão mútua

- Baseado em duas primitivas:
  - ***enter()***: entra na SC; se recurso não está disponível, bloqueia
  - ***exit()***: saída da SC; libera recurso o que provoca a eventual autorização para outro processo ingressar (desbloquear)
- Propriedades:
  - ***Safety***: no máximo um processo por vez na SC
  - ***Liveness***: os pedidos para entrada e saída da SC devem ter sucesso
    - Sem deadlock e starvation
  - ***Ordering***: se um pedido para entrar na SC aconteceu antes (*happened-before*) de outro, então a entrada na SC deve respeitar essa ordem



## Suposições:

- Processos não falham por colapso ("pendurar")
- Mensagens não são perdidas (sem falhas por omissão)



Senão, adicionar mecanismos de tolerância a falhas

# Critérios para avaliação de algoritmos

- ① Atendimento as propriedades *safety*, *liveness* e *ordering*
- ② Largura de banda consumida
  - Proporcional ao número de mensagens trocadas para realização do protocolo `enter()` e `exit()`
- ③ Cliente delay
  - Sofrido por cada processo a cada operação de `enter()` e `exit()`
- ④ Synchronization delay
  - Devido a comunicação necessária entre entradas sucessivas
  - Tempo entre um processo sair da SC e o próximo entrar na SC
- ⑤ Tolerância a falhas

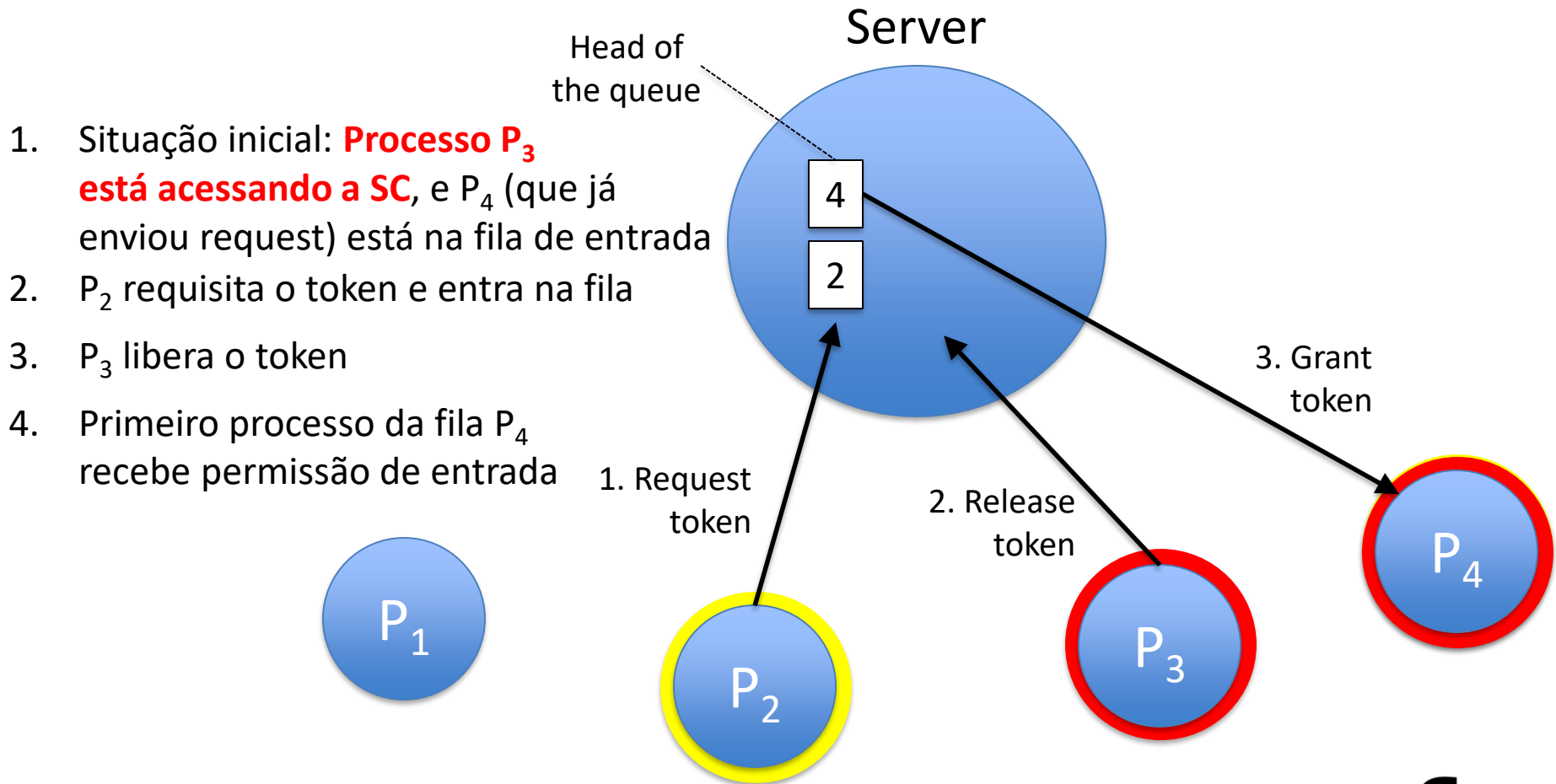
# Algoritmos para exclusão mútua em sistemas distribuídos

- Algoritmo centralizado
  - Baseado em um modelo cliente-servidor (monitor)
- Algoritmos distribuídos
  - Baseado em token de permissão (anel)
  - Baseado em fila distribuída (Ricart-Agrawala)

# Algoritmo centralizado

- Utiliza um controlador/servidor que concede permissões para que processos entrem na SC
  - Um processo é eleito para ser o controlador
    - Utilizando algum dos algoritmos (p. ex. *bully*) vistos em aulas anteriores...
  - Processo controlador decide sobre entrada na seção crítica
  - Mecanismo mais simples para alcançar exclusão mútua
- Princípio de funcionamento:
  - Baseado em três mensagens: ***request***, ***grant*** e ***release***
  - Processos pedem permissão para entrar na seção crítica (*request*):
    - Se **livre**: recebem autorização do controlador (*grant*)
    - Se **ocupado**: ficam bloqueados a espera da mensagem de *grant*
  - Processo ao liberar a SC, avisa o controlador (*release*)

# Exemplo: algoritmo centralizado



# Algoritmo centralizado: vantagens e desvantagens

- Baseado na troca de apenas três mensagens
  - Duas para o protocolo de entrada: ***request, grant***
  - Uma para o protocolo de saída: ***release***
- Propriedades:
  - *Safety*: apenas processo com token é autorizado a entrar na seção crítica
  - *Liveness*: obtida através de fila de atendimento de requisições
  - *Ordering*: satisfaz relação *happened-before*? **NÃO**
- Client delay?
  - Entrada na SC: request/grant round-trip
  - Saída da SC: caso release seja assíncrono, não há delay
- Synchronization delay?
  - Round-trip: mensagem de release para servidor, seguido de mensagem de grant
- Ponto único de falha: processo controlador
  - Recuperação exige eleição de novo controlador



# Algoritmo baseado em anel

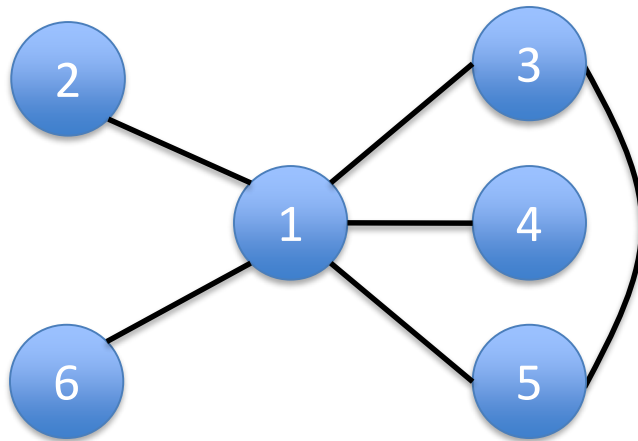
- Permite exclusão mútua entre N processos sem a necessidade de processo adicional
- Processos são logicamente organizados em anel unidirecional
  - Cada processo  $P_i$  passa um **token** para seu vizinho  $P_{(i+1) \bmod N}$
  - Ideia geral: exclusão mútua é alcançada quando processo obtém um token (ficha) que é passado de processo para processo ao redor do anel
    - Token dá ao detentor (e apenas ele) o direito de acesso à seção crítica (*safety*)
    - Sendo unidirecional elimina *deadlock*, *starvation* e é justo
- Funcionamento básico:
  - Processo que deseja entrar na seção crítica espera receber o token
    - Ao **receber o token**, entra na seção crítica
    - Ao **sair da seção crítica**, **repassa o token** adiante para o vizinho
  - Se processo **não** deseja entrar na seção crítica
    - Ao receber o token, repassa adiante para o vizinho

# Etapas do algoritmo em anel

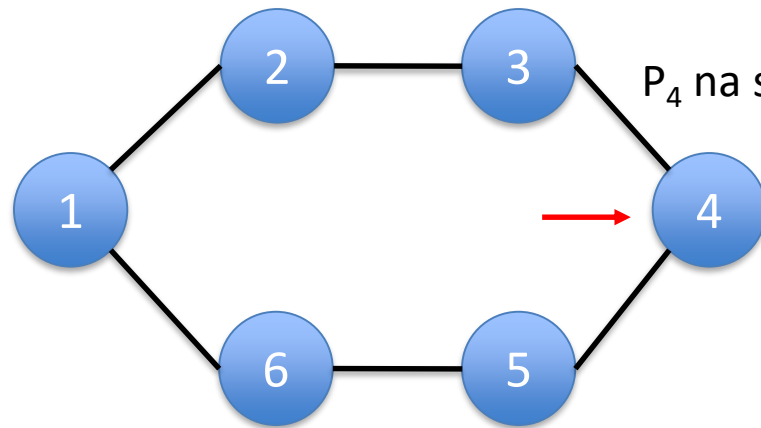
1.  $P_i$  deseja entrar na seção crítica
  - 1.1. Bloqueia a espera do token
2.  $P_j$  recebe token
  - 2.1. Se  $P_j$  deseja entrar na seção crítica, entrada é permitida e  $P_j$  segura o token
    - 2.1.1. Quando terminar de acessar a SC, repassa o token para o vizinho
  - 2.2. Se  $P_j$  não deseja usar a seção crítica, repassa o token para o vizinho

# Algoritmo em anel

- Token é uma mensagem especial que serve para obter e colher permissão para entrar na seção crítica
  - Processos esperam receber o token para entrar na seção crítica



Relação entre os processos



Lógica do anel (controle)

**Problema:** O que acontece quando nenhum processo deseja entrar na SC?

# Algoritmo baseado em anel

- Problema: algoritmo continuamente *consome bandwidth*
- Propriedades:
  - *Safety*: existe apenas um token, portanto, apenas um processo pode entrar na seção crítica
  - *Liveness*: garante que quem deseja entrar na seção crítica obterá o token em tempo finito
    - Por construção do anel, processo em algum momento receberá o token de seu vizinho
  - *Ordering*: pode não respeitar a ordem de solicitação
    - Processos podem trocar outras mensagens além do token sendo passado
- Client delay?
  - Entrada na SC: entre 0 (se acabou de receber o token) e N mensagens
  - Saída da SC: somente 1 mensagem
- Synchronization delay (entre saída de processo e entrada do próx.)?
  - Entre 1 e N mensagens

# Algoritmo Ricart-Agrawala [1981]

- Implementa exclusão mútua entre N processos
  - Baseado em multicast e relógios lógicos
  - Garante o acesso à seção crítica em uma ordem FCFS
    - Implementa uma fila distribuída
- Idéia básica:
  - Cada processo está em um de três estados possíveis:
    - **RELEASED**: processo não está na seção crítica e não deseja entrar
    - **WANTED**: processo deseja entrar na seção crítica
    - **HELD**: processo está na seção crítica
  - Processo que deseja entrar na seção crítica envia mensagem aos demais e espera que estes (todos!) autorizem sua entrada
    - Total de mensagens:  $2*(n-1)$ 
      - $n-1$  requisições de entrada e  $n-1$  respostas (autorizações)
      - Multicast nativo:  $n$  mensagens (1 de requisição e  $n-1$  autorizações)

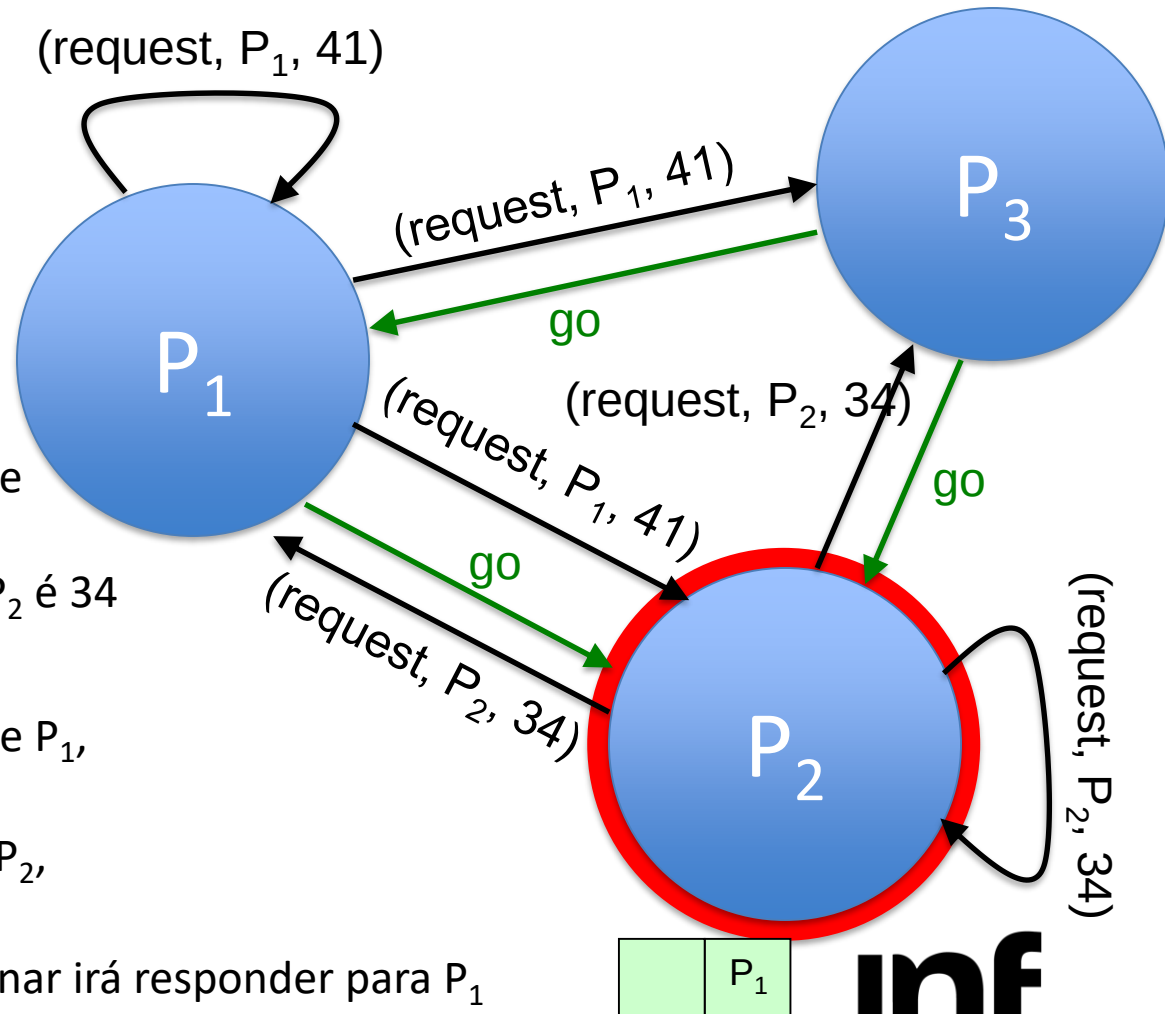
# Etapas do algoritmo Ricart-Agrawala

1.  $P_r$  **deseja entrar** na seção crítica
  - 1.1. **Envia** mensagem (multicast) de **requisição de entrada** (request,  $P_r$ , ts)
  - 1.2. **Bloqueia** a espera das **autorizações** (TODAS!!)
2.  $P_j$  **recebe mensagem** de requisição de entrada de  $P_r$ 
  - 2.1. Se  $P_j$  **não deseja entrar na seção crítica (RELEASED)**, envia “go” para  $P_r$
  - 2.2. Se  $P_j$  **deseja entrar na SC (WANTED)**, envia “go” para  $P_r$  apenas se timestamp for menor do que o de sua própria requisição, senão, enfileira a requisição de  $P_r$
  - 2.3. Se  $P_j$  **está usando a seção crítica (HELD)**, enfileira a requisição de  $P_r$
3.  $P_r$  **recebe n-1 replies** do tipo “go”: **entra na seção crítica**
4.  $P_r$  **ao sair** da seção crítica **envia “go”** para cada processo que está na sua lista de requisição pendente

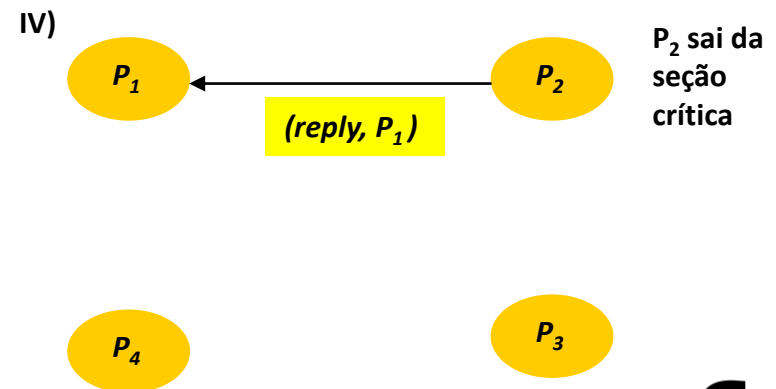
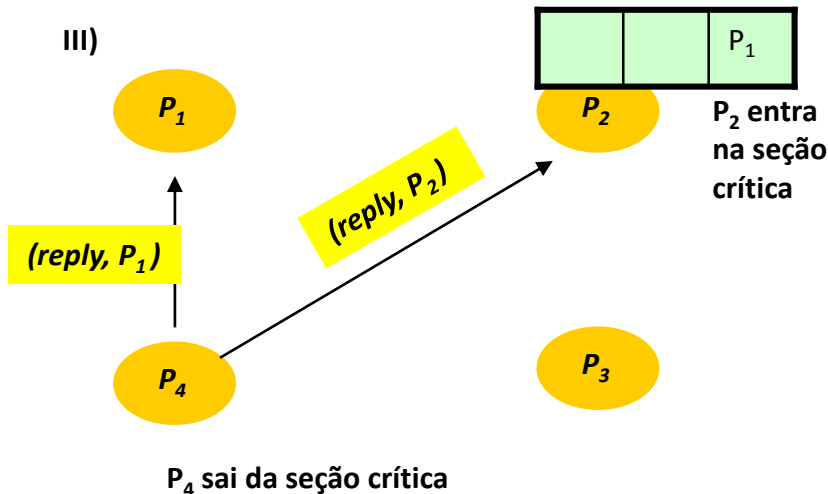
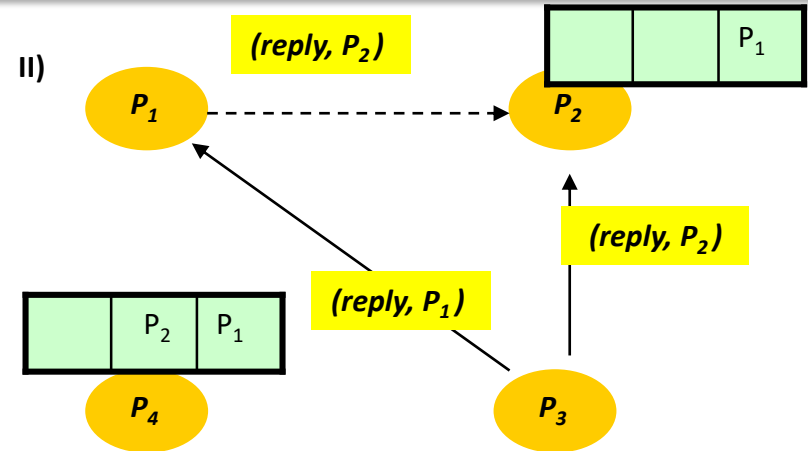
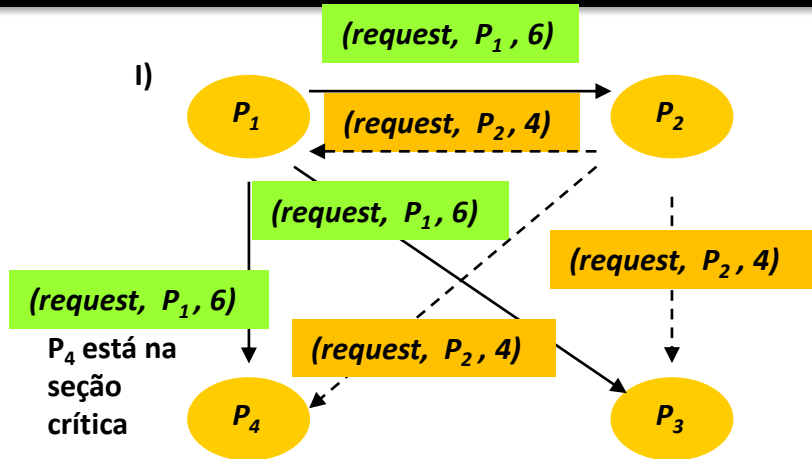
Se dois processos requisitarem acesso à SC “ao mesmo tempo”, quem irá entrar? E se os dois tiverem o mesmo timestamp?

# Exemplo: algoritmo de Ricart-Agrawala

1. Situação inicial:  $P_3$  não deseja entrar na SC
  - $P_1$  e  $P_2$  requisitam entrada de forma concorrente
  - Timestamp de  $P_1$  é 41 e de  $P_2$  é 34
2.  $P_3$  responde imediatamente
3. Quando  $P_2$  recebe requisição de  $P_1$ , não responde e enfileira
4. Quando  $P_1$  recebe requisição de  $P_2$ , responde imediatamente
5.  $P_2$  entra na SC, e quando terminar irá responder para  $P_1$



# Exemplo: algoritmo de Ricart-Agrawala





# Algoritmo Ricart-Agrawala

## *On initialization*

state := **RELEASED**;

## *To enter the critical section*

state := **WANTED**;

Multicast request to all processes;

T := request's timestamp;

Wait until (number of replies received = (N - 1));

state := **HELD**;

} request processing  
deferred here

## *On receipt of a request $\langle T_i, p_i \rangle$ at $p_j$ ( $i \neq j$ )*

if (state = **HELD** or (state = **WANTED** and  $(T, p_j) < (T_i, p_i)$ ))

then

    queue request from  $p_i$  without replying;

else

    reply immediately to  $p_i$ ;

end if

## *To exit the critical section*

state := **RELEASED**;

reply to any queued requests;

# Algoritmo Ricart-Agrawala [1981]

- Propriedades:
  - *Safety*: apenas um processo receberá as  $n-1$  autorizações
  - *Liveness*: atendimento conforme relógios lógicos, portanto, não há postergação infinita
  - *Ordering*: garantido através de relógios lógicos
- Número de mensagens?
  - Entrar na SC:  $2(N-1)$
- Client delay?
  - Round trip time
- Synchronization delay?
  - Tempo de transmissão de apenas uma mensagem

# Comentários gerais sobre algoritmos exclusão mútua em relação a tolerância a falhas

- Algoritmo centralizado
  - Perda do controlador → algoritmo de eleição
- Algoritmos baseado em anel (token)
  - Erros possíveis
    - Falhas de processos ou rede “quebram” anel
    - Possibilidade do token ser perdido
  - Processo de monitoração do anel (algoritmo de eleição)
- Algoritmos de fila distribuída (Ricart-Agrawala)
  - Possível receber a autorização apenas de um sub-conjunto de processos (Algoritmos de Maekawa, Tanenbaum, Lamport...)
- Não endereçam todos os problemas
  - Questão do estado dos recursos no momento da falha
    - Onde estava o token?

# Leituras adicionais

- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G.—  
“*Distributed Systems: Concepts and Design*” (5<sup>th</sup> edition),  
Addison Wesley, 2012
  - Capítulo 15 (seção 15.2)

# Exercícios

- ① No **algoritmo centralizado** para exclusão mútua, descreva uma situação/exemplo onde duas requisições não são processadas em ordem happened-before.
- ② Apresente um exemplo de execução do **algoritmo baseado em anel** que mostre que a entrada de processos na seção crítica não acontece, necessariamente, em ordem happened-before. *Lembre-se: a topologia de anel é apenas utilizada para controle, e processos podem se comunicar através de alguma outra interconexão física.*
- ③ Em alguns sistemas, cada processo pode tipicamente utilizar a seção crítica diversas vezes antes que outro processo requisiite acesso. Explique porque o **algoritmo de Ricart e Agrawala** é ineficiente para este caso, e descreva como melhorar a performance do algoritmo.